

СИСТЕМА НАВИГАЦИИ: МЕНЮ И ПЕРЕКЛЮЧЕНИЕ ЭКРАНОВ

МОДУЛЬ: АРХИТЕКТУРА ПРИЛОЖЕНИЯ | ТЕОРИЯ

ЧТО ТАКОЕ НАВИГАЦИЯ?

Представь, что у тебя есть несколько игр: кликер, мемогенератор, Flappy Bird. Но сейчас каждая игра — это отдельное приложение. Непривычно открывать 5 разных приложений, чтобы поиграть в каждое.

Навигация — это способ переключаться между экранами внутри одного приложения. Как меню в ресторане: ты смотришь список блюд и выбираешь то, что хочешь попробовать.

В нашем приложении мы сделаем главное меню с карточками заданий. При нажатии на карточку откроется соответствующий экран. Кнопка 'Назад' вернёт нас в меню.

КАК ЭТО РАБОТАЕТ ВНУТРИ?

Представь систему навигации как вокзал. У нас есть:

1. Маршруты (Routes) — названия платформ: 'Главная', 'Задание 1', 'Задание 2' и так далее.
2. Стек переходов (BackStack) — список всех экранов, которые мы посетили. Когда нажимаешь 'Назад', мы просто удаляем последний элемент из списка.
3. Провайдер экранов (EntryPointProvider) — диспетчер, который знает какой экран показывать для каждого маршрута.
4. Реестр экранов (ScreensRegistry) — таблица, где хранится связь: 'Задание 1 = ХакерскийТерминал', 'Задание 2 = ЭкранЗаглушка'.

ПОЧЕМУ NAVIGATION 3?

В Android есть несколько версий библиотеки навигации. Мы используем Navigation 3 — самую новую.

Она работает на всех платформах: Android, iOS, Desktop и Web. Это важно для Kotlin Multiplatform проектов.

Navigation 3 использует другой подход: вместо строковых маршрутов ('screen1', 'screen2') мы используем специальные классы-маршруты. Это безопаснее — компилятор поймает ошибку, если ты опечатаешься.

ЧТО ТАКОЕ PLACEHOLDER (ЗАГЛУШКА)?

PlaceholderScreen — это экран-заглушка. Он показывается, когда ученик ещё не создал свой экран для какого-то задания.

Это как пустая страница в тетради: место зарезервировано, но пока там ничего нет. Когда ты создашь свой экран и зарегистрируешь его — заглушка исчезнет.

СИСТЕМА НАВИГАЦИИ: МЕНЮ И ПЕРЕКЛЮЧЕНИЕ ЭКРАНОВ

МОДУЛЬ: АРХИТЕКТУРА ПРИЛОЖЕНИЯ | ID: 9

ЭТАП 1: ШАГ 1: Библиотеки в composeApp/build.gradle.kts

```
// Открой composeApp/build.gradle.kts
// В commonMain.dependencies добавь:

implementation(libs.androidx.navigation3.ui)
implementation(libs.kotlinx.serialization.core)
implementation(libs.kotlinx.serialization.json)
implementation(libs.androidx.lifecycle.viewmodelCompose)
implementation(libs.androidx.lifecycle.runtimeCompose)
implementation(libs.androidx.lifecycle.viewmodel.navigation3)
```

ЭТАП 2: ШАГ 2: Версии в gradle/libs.versions.toml

```
[versions]
androidx-nav3 = "1.1.0-alpha01"
kotlinxSerializationCore = "1.10.0"
androidx-lifecycle = "2.10.0"

[libraries]
androidx-navigation3-ui =
  { module = "org.jetbrains.androidx.navigation3:
    navigation3-ui", version.ref = "androidx-nav3" }
kotlinx-serialization-core =
  { module = "org.jetbrains.kotlinx:kotlinx-
    serialization-core", version.ref = "kotlinxSerializationCore" }
kotlinx-serialization-json =
  { module = "org.jetbrains.kotlinx:kotlinx-
    serialization-json", version.ref = "kotlinxSerializationCore" }
androidx-lifecycle-viewmodelCompose =
  { module = "org.jetbrains.androidx.lifecycle:
    lifecycle-viewmodel-compose",
    version.ref = "androidx-lifecycle" }
androidx-lifecycle-runtimeCompose =
  { module = "org.jetbrains.androidx.lifecycle:
    lifecycle-runtime-compose",
    version.ref = "androidx-lifecycle" }
androidx-lifecycle-viewmodel-navigation3 =
  { module = "org.jetbrains.androidx.lifecycle:
    lifecycle-viewmodel-navigation3",
    version.ref = "androidx-lifecycle" }
```


СИСТЕМА НАВИГАЦИИ: МЕНЮ И ПЕРЕКЛЮЧЕНИЕ ЭКРАНОВ

МОДУЛЬ: АРХИТЕКТУРА ПРИЛОЖЕНИЯ | ID: 9

ЭТАП 3: ШАГ 3: Плагин сериализации в build.gradle.kts (корень)

```
// Файл В КОРНЕ проекта: build.gradle.kts
// НЕ composeApp/build.gradle.kts, а тот что выше!

// В plugins { } добавь:
alias(libs.plugins.kotlin.serialization) apply false

// А в composeApp/build.gradle.kts в plugins { }:
alias(libs.plugins.kotlin.serialization)

// В libs.versions.toml уже должен быть плагин:
[plugins]
jetbrains-kotlin-serialization =
    { id = "org.jetbrains.kotlin.plugin.serialization",
      version.ref = "kotlin" }

// Нажми 'Sync Now' для синхронизации
```

ЭТАП 4: ШАГ 4: Маршруты — navigation/StudentRoutes.kt (импорты)

```
// Создай папку ui/navigation/
// Файл StudentRoutes.kt

package io.androidtafs.exampleproject.ui.navigation

import androidx.navigation3.runtime.NavKey
import kotlinx.serialization.Serializable
```


СИСТЕМА НАВИГАЦИИ: МЕНЮ И ПЕРЕКЛЮЧЕНИЕ ЭКРАНОВ

МОДУЛЬ: АРХИТЕКТУРА ПРИЛОЖЕНИЯ | ID: 9

ЭТАП 5: ШАГ 5: Маршруты — sealed interface (продолжение файла)

```
// @Serializable нужен навигации для сохранения маршрутов.
@Serializable
sealed interface StudentRoute : NavKey {
    // Главный экран    меню
    @Serializable
    data object Home : StudentRoute

    // Экран задания с номером
    data class Task(val id: Int) : StudentRoute
}

// Home    один экран (data object).
// Task    семейство экранов, параметр = номер.
```

ЭТАП 6: ШАГ 6: ViewModel — navigation/StudentMainViewModel.kt (часть 1)

```
// В папке navigation создай StudentMainViewModel.kt

package io.androidtafs.exampleproject.ui.navigation

import androidx.lifecycle.ViewModel
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.asStateFlow

class StudentMainViewModel : ViewModel() {
    // Стек переходов    список посещённых экранов.
    private val _backStack =
        MutableStateFlow<List<StudentRoute>>(
            listOf(StudentRoute.Home))
    val backStack = _backStack.asStateFlow()
```


СИСТЕМА НАВИГАЦИИ: МЕНЮ И ПЕРЕКЛЮЧЕНИЕ ЭКРАНОВ

МОДУЛЬ: АРХИТЕКТУРА ПРИЛОЖЕНИЯ | ID: 9

ЭТАП 7: ШАГ 7: ViewModel — методы навигации (часть 2 файла)

```
// Перейти на экран
fun navigateTo(route: StudentRoute) {
    val current = _backStack.value.toMutableList()
    if (route is StudentRoute.Home) {
        current.clear() // Домой чистим стек
    }
    current.add(route)
    _backStack.value = current
}

// Назад удаляем последний экран
fun goBack() {
    val current = _backStack.value.toMutableList()
    if (current.size > 1) {
        current.removeAt(current.lastIndex)
        _backStack.value = current
    }
}
}
```

ЭТАП 8: ШАГ 8: Заглушка — screens/PlaceholderScreen.kt (импорты)

```
// Создай папку ui/screens/

package io.androidtafs.exampleproject.ui.screens

import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
```


СИСТЕМА НАВИГАЦИИ: МЕНЮ И ПЕРЕКЛЮЧЕНИЕ ЭКРАНОВ

МОДУЛЬ: АРХИТЕКТУРА ПРИЛОЖЕНИЯ | ID: 9

ЭТАП 9: ШАГ 9: Заглушка — composable-функция (продолжение)

```
@Composable
fun PlaceholderScreen(taskId: Int) {
    Box(
        modifier = Modifier.fillMaxSize(),
        contentAlignment = Alignment.Center
    ) {
        Column(
            horizontalAlignment =
                Alignment.CenterHorizontally,
            verticalArrangement =
                Arrangement.spacedBy(8.dp)
        ) {
            Text(
                text = "Задание $taskId",
                style =
                    MaterialTheme.typography.
                        headlineMedium,
                fontWeight = FontWeight.Bold,
                color = MaterialTheme.colorScheme.onSurface
            )
            Text(
                text = "Экран ещё не создан.\n
Создай свой экран!",
                style =
                    MaterialTheme.typography.
                        bodyLarge,
                color =
                    MaterialTheme.colorScheme.
                        onSurfaceVariant,
                modifier = Modifier.padding(
                    horizontal = 32.dp
                )
            )
        }
    }
}
```


СИСТЕМА НАВИГАЦИИ: МЕНЮ И ПЕРЕКЛЮЧЕНИЕ ЭКРАНОВ

МОДУЛЬ: АРХИТЕКТУРА ПРИЛОЖЕНИЯ | ID: 9

ЭТАП 10: ШАГ 10: Реестр — screens/StudentScreensRegistry.kt

```
// В папке screens создай StudentScreensRegistry.kt

package io.androidtafs.exampleproject.ui.screens

import androidx.compose.runtime.Composable

object StudentScreensRegistry {
    // Словарь: номер задания -> экран
    private val screens =
        mutableMapOf<Int,
            @Composable () -> Unit>()

    // Зарегистрировать экран
    fun register(taskId: Int,
        screen: @Composable () -> Unit) {
        screens[taskId] = screen
    }

    // Получить экран или заглушку
    @Composable
    fun getScreen(taskId: Int):
        @Composable () -> Unit {
        return screens[taskId]
            ?: { PlaceholderScreen(taskId) }
    }
}
```


СИСТЕМА НАВИГАЦИИ: МЕНЮ И ПЕРЕКЛЮЧЕНИЕ ЭКРАНО

МОДУЛЬ: АРХИТЕКТУРА ПРИЛОЖЕНИЯ | ID: 9

ЭТАП 11: ШАГ 11: Меню — feature/main/StudentNav.kt (импорты)

```
// Создай папку ui/feature/main/

package io.androidtafs.exampleproject.ui.feature.main

import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.grid.GridCells
import androidx.compose.foundation.lazy.grid.
    LazyVerticalGrid
import androidx.compose.foundation.lazy.grid.items
import androidx.compose.foundation.shape.
    RoundedCornerShape
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
```


СИСТЕМА НАВИГАЦИИ: МЕНЮ И ПЕРЕКЛЮЧЕНИЕ ЭКРАНО

МОДУЛЬ: АРХИТЕКТУРА ПРИЛОЖЕНИЯ | ID: 9

ЭТАП 12: ШАГ 12: Меню — data class + начало функции

```
// Данные карточки в меню
data class StudentTaskEntry(
    val id: Int,           // Номер
    val title: String,     // Название
    val icon: String       // Эмодзи
)

@Composable
fun StudentNav(onTaskSelected: (Int) -> Unit) {
    val tasks = listOf(
        StudentTaskEntry(1, "Хак. терминал", "💻"),
        StudentTaskEntry(3, "Генератор мемов", "😄"),
        StudentTaskEntry(4, "Кибер-охотник", "🕵️"),
        StudentTaskEntry(5, "Квант. реактор", "☀️"),
        StudentTaskEntry(6, "Подв. бездна", "🕳️"),
        StudentTaskEntry(7, "Flappy Bird", "🐦"),
        StudentTaskEntry(8, "Визуал. роман", "📖")
    )

    Surface(color = Color(0xFF0F0F1A)) {
        Column(modifier = Modifier.fillMaxSize()) {
```


СИСТЕМА НАВИГАЦИИ: МЕНЮ И ПЕРЕКЛЮЧЕНИЕ ЭКРАНОВ

МОДУЛЬ: АРХИТЕКТУРА ПРИЛОЖЕНИЯ | ID: 9

ЭТАП 13: ШАГ 13: Меню — заголовок + сетка карточек

```
Text(
    text = "Меню заданий",
    modifier = Modifier
        .fillMaxWidth()
        .padding(vertical = 24.dp),
    color = Color.White,
    fontWeight = FontWeight.Black,
    fontSize = 28.sp,
    textAlign =
        androidx.compose.ui.text.
            style.TextAlign.Center
)

LazyVerticalGrid(
    columns = GridCells.Fixed(2),
    horizontalArrangement =
        Arrangement.spacedBy(12.dp),
    verticalArrangement =
        Arrangement.spacedBy(12.dp),
    contentPadding =
        PaddingValues(16.dp)
) {
    items(tasks, key = { it.id }) {
```


СИСТЕМА НАВИГАЦИИ: МЕНЮ И ПЕРЕКЛЮЧЕНИЕ ЭКРАНОВ

МОДУЛЬ: АРХИТЕКТУРА ПРИЛОЖЕНИЯ | ID: 9

ЭТАП 14: ШАГ 14: Меню — карточка задания (продолжение)

```
task ->
Card(
    modifier = Modifier
        .fillMaxWidth()
        .clip(RoundedCornerShape(16.dp))
        .clickable {
            onTaskSelected(task.id)
        },
    shape =
        RoundedCornerShape(16.dp),
    elevation =
        CardDefaults.cardElevation(
            defaultElevation = 4.dp),
    colors =
        CardDefaults.cardColors(
            containerColor = Color.Black)
) {
    Box(
        modifier = Modifier
            .fillMaxWidth()
            .padding(20.dp),
        contentAlignment =
            Alignment.Center
    ) {
```


СИСТЕМА НАВИГАЦИИ: МЕНЮ И ПЕРЕКЛЮЧЕНИЕ ЭКРАНОВ

МОДУЛЬ: АРХИТЕКТУРА ПРИЛОЖЕНИЯ | ID: 9

ЭТАП 15: ШАГ 15: Меню — содержимое карточки + закрытие

```
Column(  
    horizontalAlignment =  
        Alignment.CenterHorizontally,  
    verticalArrangement =  
        Arrangement.Center  
) {  
    Text(  
        text = task.icon,  
        fontSize = 48.sp,  
        modifier = Modifier.padding(  
            bottom = 12.dp))  
    Text(  
        text = task.title,  
        color = Color.White,  
        fontWeight =  
            FontWeight.Bold,  
        fontSize = 14.sp)  
    }  
    }  
    }  
    }  
    }  
    }  
    }
```


СИСТЕМА НАВИГАЦИИ: МЕНЮ И ПЕРЕКЛЮЧЕНИЕ ЭКРАНОВ

МОДУЛЬ: АРХИТЕКТУРА ПРИЛОЖЕНИЯ | ID: 9

ЭТАП 16: ШАГ 16: Провайдер — navigation/StudentEntryProvider.kt (импорты)

```
// Главный диспетчер экранов

package io.androidtafs.exampleproject.ui.navigation

import androidx.compose.foundation.layout.padding
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.automirrored.
    rounded.ArrowBack
import androidx.compose.material3.*
import androidx.compose.runtime.Composable
import androidx.compose.runtime.remember
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
import androidx.navigation3.runtime.NavEntry
import androidx.navigation3.runtime.NavKey
import io.androidtafs.exampleproject.ui.feature.main.
    StudentNav
import io.androidtafs.exampleproject.ui.screens.
    StudentScreensRegistry
```

ЭТАП 17: ШАГ 17: Провайдер — функция + маршрут Home

```
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun rememberStudentEntryProvider(
    viewModel: StudentMainViewModel
): (NavKey) -> NavEntry<NavKey> =
    remember(viewModel) {
        { key ->
            NavEntry(key) {
                when (key) {
                    is StudentRoute.Home -> {
                        StudentNav(
                            onTaskSelected = { taskId ->
                                viewModel.navigateTo(
                                    StudentRoute.Task(taskId))
                            })
                    }
                }
            }
        }
    }
```


СИСТЕМА НАВИГАЦИИ: МЕНЮ И ПЕРЕКЛЮЧЕНИЕ ЭКРАНО

МОДУЛЬ: АРХИТЕКТУРА ПРИЛОЖЕНИЯ | ID: 9

ЭТАП 18: ШАГ 18: Провайдер — маршрут Task (Scaffold)

```
is StudentRoute.Task -> {
    val taskScreen =
        StudentScreensRegistry.
            getScreen(key.id)

    Scaffold(
        topBar = {
            Surface(
                shadowElevation = 4.dp) {
                CenterAlignedTopAppBar(
                    title = {
                        Text(
                            "Задание  ${key.id}")
                    },
                    navigationIcon = {
```


СИСТЕМА НАВИГАЦИИ: МЕНЮ И ПЕРЕКЛЮЧЕНИЕ ЭКРАНА

МОДУЛЬ: АРХИТЕКТУРА ПРИЛОЖЕНИЯ | ID: 9

ЭТАП 19: ШАГ 19: Провайдер — кнопка Назад + контент экрана

```

        IconButton(
            onClick = {
                viewModel.goBack()
            }) {
            Icon(
                imageVector =
                    Icons.AutoMirrored.
                    Rounded.ArrowBack,
                contentDescription =
                    "Назад"
            )
        }
    },
    colors =
        TopAppBarDefaults.
        topAppBarColors(
            containerColor =
MaterialTheme.colorScheme.surface
        )
    )
}
},
containerColor =
    MaterialTheme.colorScheme.background
) { paddingValues ->

```


СИСТЕМА НАВИГАЦИИ: МЕНЮ И ПЕРЕКЛЮЧЕНИЕ ЭКРАНО

МОДУЛЬ: АРХИТЕКТУРА ПРИЛОЖЕНИЯ | ID: 9

ЭТАП 20: ШАГ 20: Провайдер — отрисовка экрана + закрытие

```
                Surface(
                    modifier = Modifier.padding(paddingValues)
                ) {
                    taskScreen()
                }
            }
        }
    }
    else -> {}
}
}
```

ЭТАП 21: ШАГ 21: App.kt — импорты (feature/main/App.kt)

```
// Главный composable приложения

package io.androidtafs.exampleproject.ui.feature.main

import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.runtime.Composable
import androidx.compose.runtime.saveable.
    rememberSaveableStateHolder
import androidx.compose.ui.Modifier
import androidx.lifecycle.compose.
    collectAsStateWithLifecycle
import androidx.lifecycle.viewmodel.compose.viewModel
import androidx.navigation3.runtime.
    SaveableStateHolderNavEntryDecorator
import androidx.navigation3.ui.NavDisplay
import io.androidtafs.exampleproject.ui.navigation.
    StudentMainViewModel
import io.androidtafs.exampleproject.ui.navigation.
    rememberStudentEntryProvider
```


СИСТЕМА НАВИГАЦИИ: МЕНЮ И ПЕРЕКЛЮЧЕНИЕ ЭКРАНОВ

МОДУЛЬ: АРХИТЕКТУРА ПРИЛОЖЕНИЯ | ID: 9

ЭТАП 22: ШАГ 22: App.kt — функция App (начало)

```
@Composable
fun App() {
    val viewModel: StudentMainViewModel = viewModel()

    // Наблюдаем за стеком навигации.
    // Когда стек изменится экран перерисуеться.
    val backStack =
        viewModel.backStack.collectAsStateWithLifecycle().value

    val entryProvider =
        rememberStudentEntryProvider(viewModel)

    // Сохраняет состояние экранов при переходах
    val saveableStateHolder =
        rememberSaveableStateHolder()
```

ЭТАП 23: ШАГ 23: App.kt — NavDisplay (продолжение)

```
MaterialTheme {
    Surface(
        modifier = Modifier.fillMaxSize(),
        color = MaterialTheme.colorScheme.background
    ) {
        if (backStack.isNotEmpty()) {
            NavDisplay(
                backStack = backStack,
                onBack = { viewModel.goBack() },
                entryDecorators = listOf(
                    SaveableStateHolderNavEntryDecorator(
                        saveableStateHolder
                    ),
                ),
                entryProvider = entryProvider
            )
        }
    }
}
```


СИСТЕМА НАВИГАЦИИ: МЕНЮ И ПЕРЕКЛЮЧЕНИЕ ЭКРАНОВ

МОДУЛЬ: АРХИТЕКТУРА ПРИЛОЖЕНИЯ | ID: 9

ЭТАП 24: ШАГ 24: MainActivity — регистрация экранов

```
// Открой androidMain/MainActivity.kt

package io.androidtafs.exampleproject

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import io.androidtafs.exampleproject.ui.feature.main.App
import io.androidtafs.exampleproject.ui.screens.
    StudentScreensRegistry

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        enableEdgeToEdge()
        super.onCreate(savedInstanceState)
    }
}
```

ЭТАП 25: ШАГ 25: MainActivity — регистрация + запуск App

```
        // === РЕГИСТРИРУЕМ СВОИ ЭКРАНЫ ===
        // StudentScreensRegistry.register(1, {
        //     HackerTerminalScreen()
        // })
        // StudentScreensRegistry.register(4, {
        //     CyberBountyHunterScreen()
        // })

        setContent {
            App()
        }
    }
}
```


СИСТЕМА НАВИГАЦИИ: МЕНЮ И ПЕРЕКЛЮЧЕНИЕ ЭКРАНОВ

МОДУЛЬ: АРХИТЕКТУРА ПРИЛОЖЕНИЯ | ID: 9

ЭТАП 26: ШАГ 26: Проверка и запуск!

```
// Запусти приложение. Ты должен увидеть:  
  
// 1. Тёмное меню с карточками заданий  
// 2. При нажатии экран 'Задание N'  
//    с кнопкой Назад сверху  
// 3. Если экран не зарегистрирован заглушка:  
//    'Экран ещё не создан'  
  
// Чтобы свой экран появился вместо заглушки,  
// раскомментируй строку register в MainActivity:  
// StudentScreensRegistry.register(1, {  
//     ТвойЭкран()  
// })
```

ПОЗДРАВЛЯЕМ! Навигация готова!

- Главное меню с карточками
- Кнопка Назад на каждом экране
- Реестр экранов (легко добавлять новые)
- Заглушки для незагруженных заданий

ДОПОЛНИТЕЛЬНОЕ ЗАДАНИЕ:

Добавь в меню свой собственный экран! Придумай название, выбери эмодзи-иконку и зарегистрируй его в реестре. Попробуй добавить 2 своих экрана.